Universidad Popular del Cesar

Ingeniería de Sistemas
Universidad Popular del Cesar

FACULTAD DE INGENIERÍA Y TECNOLÓGICAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA DE TALLER · 4TA UNIDAD

Seguridad de la API con JWT

Registro, inicio de sesión y endpoints protegidos con JSON Web Tokens

Asignatura	Programación Web
Unidad	4ta Unidad — Seguridad en APIs
Docente	Ing. Sadainer Hernández Chacón
Proyecto base	WebApi2026New (ASP.NET Core · .NET 10 · SQL Server)
Herramienta de prueba	Scalar

Universidad Popular del Cesar
Valledupar, Cesar · 2026

Propósito

En la unidad anterior construiste la API de Personas y Direcciones. Hoy cualquiera puede consultarla, modificarla o borrar datos, y las contraseñas se guardan en texto plano. En este taller la vas a proteger con **JSON Web Tokens (JWT)**.

Resultados esperados

Al terminar el taller tu API debe:

1. Registrar personas guardando la contraseña como **hash**, nunca en texto plano.
2. Iniciar sesión y devolver un **token JWT**.
3. Responder en `/api/Personas` y `/api/Direcciones` **solo si se envía un token válido**.
4. Poder probarse desde **Scalar**, pegando el token en el campo de autenticación.

Antes de empezar

- Tu proyecto **WebApi2026New** de la unidad 3 debe compilar.
- La base de datos debe estar creada en la instancia y con las migraciones al día. **Haz primero el Paso 0.**
- Ten a mano Visual Studio 2026 (o el SDK de .NET 10) y SQL Server LocalDB.
- Haz una copia de tu proyecto o un *commit* antes de empezar.

Este taller no usa roles: cualquier persona autenticada puede usar todos los endpoints protegidos. Tampoco cambia el modelo `Persona`, así que **no necesitas crear migraciones nuevas**, pero la base debe quedar actualizada en el Paso 0.

Archivos que se tocan

Archivo	Acción
`WebApi2026New.csproj`	+ 2 paquetes
`appsettings.json` / `appsettings.Development.json`	+ sección `Jwt`
`DTOs/AuthDTO.cs`	**nuevo**
`DTOs/PersonaDTO.cs`	+ `PersonaUpdateDTO`
`Helpers/JwtHelper.cs`	**nuevo**
`Program.cs`	configurar JWT y Scalar
`Controllers/AuthController.cs`	**nuevo**
`Controllers/PersonasController.cs`	`[Authorize]`, hash en `POST`, `PUT` con DTO
`Controllers/DireccionController.cs`	`[Authorize]`

PASO 0 Preparar la base de datos y las migraciones

Antes de tocar el código del JWT, comprueba que la base de datos existe en la instancia y que coincide con tus migraciones. Si este paso falla, todo lo demás fallará también.

0.1 Verificar la instancia

1. Revisa en `appsettings.json` la cadena `DefaultConnection`: la instancia es `(localdb)\MSSQLLocalDB` y la base es `MyDB2026`.
2. Comprueba que la instancia existe y está iniciada. En una terminal:

```
sqllocaldb info
sqllocaldb start MSSQLLocalDB
```

3. En Visual Studio abre **Ver → Explorador de objetos de SQL Server**, expande `(localdb)\MSSQLLocalDB → Bases de datos` y confirma que existe `MyDB2026` con las tablas `Personas`, `Direcciones` y `__EFMigrationsHistory`.

0.2 Revisar las migraciones

Todos los comandos de este paso se ejecutan en la **Consola del Administrador de paquetes** de Visual Studio (**Herramientas → Administrador de paquetes NuGet → Consola del Administrador de paquetes**). En la lista *Proyecto predeterminado* de la consola debe estar seleccionado `WebApi2026New`.

Lista las migraciones y cuáles están aplicadas:

```
Get-Migration
```

Compara lo que aparece con la carpeta `Migrations` del proyecto y con la tabla `__EFMigrationsHistory` de la base de datos.

0.3 Ajustar según tu caso

Situación	Qué hacer
Todas las migraciones aparecen aplicadas	Nada; pasa al punto 0.4
Hay migraciones sin aplicar	`Update-Database`
La última migración está mal y **no** se ha aplicado	`Remove-Migration` y vuelve a crearla con `Add-Migration`
La base `MyDB2026` no existe	`Update-Database` la crea a partir de las migraciones
No existe la carpeta `Migrations`, o `Update-Database` falla con *"There is already an object named 'Personas'"*	Reinicia las migraciones (ver abajo)

Reiniciar las migraciones (borra los datos de prueba de la base):

1. Elimina la carpeta `Migrations` del proyecto si existe.
2. Ejecuta en la consola:

```
Drop-Database
Add-Migration Inicial
Update-Database
```

3. `Drop-Database` pide confirmación: escribe `A` (o `Y`) y presiona Enter.

0.4 Comprobar

Ejecuta la API **antes** de agregar el JWT y prueba `GET /api/Personas` en Scalar. Debe responder `200`. Si responde `500`, revisa el mensaje: casi siempre es la cadena de conexión o una migración pendiente.

> ⚠ No sigas al Paso 1 hasta que este punto funcione. Así, si algo falla después, sabrás que el problema está en el JWT y no en la base de datos.

PASO 1 Instalar los paquetes

En la carpeta del proyecto (`WebApi2026New/WebApi2026New`):

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 10.0.13
dotnet add package BCrypt.Net-Next --version 4.0.3
```

O en Visual Studio: **clic derecho en el proyecto → Administrar paquetes NuGet → Examinar**.

- **JwtBearer**: valida el token en cada petición y trae las clases para crearlo.
- **BCrypt.Net-Next**: calcula el hash de las contraseñas.

WebApi2026New.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="BCrypt.Net-Next" Version="4.0.3" />
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.13"
/>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.13" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.13" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.13" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.13">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Scalar.AspNetCore" Version="2.18.2" />
    <PackageReference Include="Swashbuckle.AspNetCore.SwaggerUI" Version="10.3.1" />
    <PackageReference Include="System.Security.Cryptography.Pkcs" Version="10.0.13" />
  </ItemGroup>

</Project>
```

PASO 2 Configurar el JWT en appsettings

En `appsettings.json` agrega la sección `Jwt`. La `Key` queda vacía a propósito:

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=MyDB2026;Trusted_Connection=true;TrustServerCertificate=
true;",
    "AppDBContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDBContext;Trusted_Connection=True;MultipleActiveResul
tSets=true"
  },
  "Jwt": {
    "Key": "",
    "Issuer": "WebApi2026New",
    "Audience": "WebApi2026New.Clientes",
    "ExpiresMinutes": 60
  }
}
```

En `appsettings.Development.json`, que en Visual Studio aparece **dentro** de `appsettings.json` al desplegar la flechita ▸, va la clave de desarrollo:

appsettings.Development.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "Jwt": {
    "Key": "SOLO-DESARROLLO-cambia-esta-clave-por-una-aleatoria-de-64-caracteres"
  }
}
```

- `Key`: clave secreta con la que se firma el token. Debe tener **32 caracteres o más**.
- `Issuer` y `Audience`: quién emite el token y para quién es. Se validan en cada petición.
- `ExpiresMinutes`: cuánto dura el token.

> ⚠ Nunca subas la clave real de producción al repositorio. En el servidor se define con la variable de entorno `Jwt__Key`.

PASO 3 DTOs de registro y login

Crea `DTOs/AuthDTO.cs`:

DTOs/AuthDTO.cs

```
using System.ComponentModel.DataAnnotations;

namespace WebApi2026New.DTOs
{
    public class RegistroDTO
```

```
    {
        [Required]
        public string nombre { get; set; } = string.Empty;

        [Required]
        public string apellido { get; set; } = string.Empty;

        public int edad { get; set; }

        [Required, EmailAddress]
        public string correo { get; set; } = string.Empty;

        [Required, MinLength(8)]
        public string password { get; set; } = string.Empty;
    }

    public class LoginDTO
    {
        [Required, EmailAddress]
        public string correo { get; set; } = string.Empty;

        [Required]
        public string password { get; set; } = string.Empty;
    }

    public class LoginRespuestaDTO
    {
        public string token { get; set; } = string.Empty;
        public DateTime expira { get; set; }
        public int idPersona { get; set; }
        public string nombre { get; set; } = string.Empty;
    }
}
```

En `DTOs/PersonaDTO.cs` agrega este DTO de edición. Así el `PUT` ya no recibe la entidad completa y no puede sobrescribir el hash de la contraseña:

```
public class PersonaUpdateDTO
{
    public string nombre { get; set; } = string.Empty;
    public string apellido { get; set; } = string.Empty;
    public int edad { get; set; }
}
```

PASO 4 Crear `JwtHelper`

Crea la carpeta `Helpers` y dentro `JwtHelper.cs`. Esta clase arma el token con el id, el correo y el nombre de la persona, y lo firma con la clave.

Helpers/JwtHelper.cs

```
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;
using WebApi2026New.Models;

namespace WebApi2026New.Helpers
```

```
{
    // Se llena desde la sección "Jwt" de appsettings.json
    public class JwtSettings
    {
        public string Key { get; set; } = string.Empty;
        public string Issuer { get; set; } = string.Empty;
        public string Audience { get; set; } = string.Empty;
        public int ExpiresMinutes { get; set; } = 60;
    }

    public class JwtHelper
    {
        private readonly JwtSettings _settings;

        public JwtHelper(IOptions<JwtSettings> options)
        {
            _settings = options.Value;
        }

        public (string token, DateTime expira) GenerarToken(Persona persona)
        {
            var expira = DateTime.UtcNow.AddMinutes(_settings.ExpiresMinutes);

            // Payload mínimo. Nunca la contraseña.
            var claims = new List<Claim>
            {
                new(JwtRegisteredClaimNames.Sub, persona.idPersona.ToString()),
                new(JwtRegisteredClaimNames.Email, persona.correo),
                new(JwtRegisteredClaimNames.Name, $"{persona.nombre} {persona.apellido}")
            };

            var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));

            var descriptor = new SecurityTokenDescriptor
            {
                Subject = new ClaimsIdentity(claims),
                Expires = expira,
                Issuer = _settings.Issuer,
                Audience = _settings.Audience,
                SigningCredentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
            };

            var token = new JsonWebTokenHandler().CreateToken(descriptor);
            return (token, expira);
        }
    }
}
```

PASO 5 Configurar `Program.cs`

Qué hace cada bloque:

- **JWT**: lee la configuración, registra `JwtHelper` y le dice a la API cómo validar el token (firma, emisor, audiencia y expiración).
- **OpenAPI**: declara el esquema de seguridad **Bearer**. Sin esto, Scalar no muestra dónde pegar el token.
- **Scalar**: `AddPreferredSecuritySchemes("Bearer")` deja Bearer seleccionado por defecto.

- `UseAuthentication()` **antes** de `UseAuthorization()`.

Reemplaza `Program.cs` completo por:

Program.cs

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi;
using Scalar.AspNetCore;
using WebApi2026New.Data;
using WebApi2026New.Helpers;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddDbContext<AppDBContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});

// ---------- JWT ----------
builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("Jwt"));
builder.Services.AddSingleton<JwtHelper>();

var jwt = builder.Configuration.GetSection("Jwt").Get<JwtSettings>()
          ?? throw new InvalidOperationException("Falta la sección 'Jwt' en appsettings.json.");

if (Encoding.UTF8.GetByteCount(jwt.Key) < 32)
    throw new InvalidOperationException("Jwt:Key debe tener al menos 32 caracteres.");

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.MapInboundClaims = false; // conserva "sub", "email", "name"
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = jwt.Issuer,
            ValidateAudience = true,
            ValidAudience = jwt.Audience,
            ValidateLifetime = true,
            ClockSkew = TimeSpan.FromMinutes(1),
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Key)),
            NameClaimType = "name"
        };
    });

builder.Services.AddAuthorization();
// -------------------------

builder.Services.AddControllers();

// OpenAPI: declara el esquema "Bearer" para que Scalar muestre dónde pegar el token
builder.Services.AddOpenApi(options =>
{
    options.AddDocumentTransformer((document, context, cancellationToken) =>
    {
```

```
        document.Components ??= new OpenApiComponents();
        document.Components.SecuritySchemes ??= new Dictionary<string, IOpenApiSecurityScheme>();
        document.Components.SecuritySchemes["Bearer"] = new OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.Http,
            Scheme = "bearer",
            BearerFormat = "JWT",
            Description = "Pega aquí el token que devuelve POST /api/Auth/login"
        };
        document.Security =
        [
            new OpenApiSecurityRequirement
            {
                [new OpenApiSecuritySchemeReference("Bearer", document)] = []
            }
        ];
        return Task.CompletedTask;
    });
});

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference(options =>
    {
        options.Title = "WebApi2026New";
        options.AddPreferredSecuritySchemes("Bearer"); // Scalar selecciona Bearer por defecto
    });
}

app.UseHttpsRedirection();

// El orden importa: primero autenticación, luego autorización
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

PASO 6 Crear `AuthController`

Crea `Controllers/AuthController.cs` con tres endpoints:

Endpoint	Token	Qué hace
`POST /api/Auth/registro`	No	Crea la persona con la contraseña en hash
`POST /api/Auth/login`	No	Verifica correo y contraseña y devuelve el token
`GET /api/Auth/yo`	Sí	Muestra qué datos trae el token

`Controllers/AuthController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
```

```
using WebApi2026New.Data;
using WebApi2026New.DTOs;
using WebApi2026New.Helpers;
using WebApi2026New.Models;

[Route("api/[controller]")]
[ApiController]
public class AuthController : ControllerBase
{
    private readonly AppDBContext _context;
    private readonly JwtHelper _jwt;

    public AuthController(AppDBContext context, JwtHelper jwt)
    {
        _context = context;
        _jwt = jwt;
    }

    // POST: api/Auth/registro  (público)
    [HttpPost("registro")]
    public async Task<ActionResult<PersonaReadDTO>> Registro(RegistroDTO dto)
    {
        var correo = dto.correo.Trim().ToLowerInvariant();

        if (await _context.Personas.AnyAsync(p => p.correo == correo))
            return Conflict(new { mensaje = "El correo ya está registrado." });

        var persona = new Persona
        {
            nombre = dto.nombre,
            apellido = dto.apellido,
            edad = dto.edad,
            correo = correo,
            password = BCrypt.Net.BCrypt.HashPassword(dto.password) // se guarda el HASH
        };

        _context.Personas.Add(persona);
        await _context.SaveChangesAsync();

        return Ok(new PersonaReadDTO
        {
            idPersona = persona.idPersona,
            nombre = persona.nombre,
            apellido = persona.apellido,
            edad = persona.edad,
            correo = persona.correo
        });
    }

    // POST: api/Auth/login  (público)
    [HttpPost("login")]
    public async Task<ActionResult<LoginRespuestaDTO>> Login(LoginDTO dto)
    {
        var correo = dto.correo.Trim().ToLowerInvariant();
        var persona = await _context.Personas.FirstOrDefaultAsync(p => p.correo == correo);

        // Mensaje genérico: no revelar si falló el correo o la contraseña
        if (persona is null || !VerificarPassword(dto.password, persona.password))
            return Unauthorized(new { mensaje = "Credenciales inválidas." });

        var (token, expira) = _jwt.GenerarToken(persona);
```

```
        return Ok(new LoginRespuestaDTO
        {
            token = token,
            expira = expira,
            idPersona = persona.idPersona,
            nombre = $"{persona.nombre} {persona.apellido}"
        });
    }

    // GET: api/Auth/yo  (requiere token) -> muestra lo que trae el token
    [HttpGet("yo")]
    [Authorize]
    public IActionResult Yo()
    {
        return Ok(new
        {
            idPersona = User.FindFirst("sub")?.Value,
            correo = User.FindFirst("email")?.Value,
            nombre = User.Identity?.Name
        });
    }

    private static bool VerificarPassword(string password, string hashGuardado)
    {
        try
        {
            return BCrypt.Net.BCrypt.Verify(password, hashGuardado);
        }
        catch (BCrypt.Net.SaltParseException)
        {
            // La fila tiene una contraseña antigua en texto plano
            return false;
        }
    }
}
```

PASO 7 Proteger los controladores

Solo hay que agregar `[Authorize]` encima de cada clase, y su `using`:

```
using Microsoft.AspNetCore.Authorization;

[Route("api/[controller]")]
[ApiController]
[Authorize] // todos los endpoints de este controlador exigen token
public class PersonasController : ControllerBase
```

Haz lo mismo en `DireccionController.cs` (clase `DireccionesController`).

En `PersonasController` hay además dos cambios:

a) El `POST` guarda el hash en lugar del texto plano:

```
password = BCrypt.Net.BCrypt.HashPassword(dto.password) // ahora sí: hash
```

b) El `PUT` recibe `PersonaUpdateDTO`:

```
// PUT: api/Persona/5  -> recibe un DTO, no la entidad (así no se sobrescribe el password)
[HttpPut("{id}")]
public async Task<IActionResult> PutPersona(int id, PersonaUpdateDTO dto)
{
    var persona = await _context.Personas.FindAsync(id);
    if (persona == null)
        return NotFound();

    persona.nombre = dto.nombre;
    persona.apellido = dto.apellido;
    persona.edad = dto.edad;

    await _context.SaveChangesAsync();
    return NoContent();
}
```

El método `PersonaExists` ya no se usa y se puede borrar.

Así queda el controlador completo:

Controllers/PersonasController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebApi2026New.Data;
using WebApi2026New.DTOs;
using WebApi2026New.Models;

[Route("api/[controller]")]
[ApiController]
[Authorize] // todos los endpoints de este controlador exigen token
public class PersonasController : ControllerBase
{
    private readonly AppDBContext _context;
    public PersonasController(AppDBContext context)
    {
        _context = context;
    }

    // GET: api/Persona
    [HttpGet]
    public async Task<ActionResult<IEnumerable<PersonaListaDto>>> GetPersonas()
    {
        return await _context.Personas.Select(p => new PersonaListaDto
        {
            idPersona = p.idPersona,
            nombre = p.nombre,
            apellido = p.apellido,
            correo = p.correo,
            edad = p.edad,
            totalDirecciones = p.Direcciones.Count

        }).ToListAsync();
    }

    // GET: api/Persona/5
    [HttpGet("{id}")]
    public async Task<ActionResult<PersonaDTO>> GetPersona(int id)
    {
        var persona = await _context.Personas
            .Where(p => p.idPersona == id)
            .Select(p => new PersonaDTO
```

```
            {
                idPersona = p.idPersona,
                nombre = p.nombre,
                apellido = p.apellido,
                correo = p.correo,
                direcciones = p.Direcciones.Select(d => new DireccionDTO
                {
                    idDireccion = d.idDireccion,
                    calle = d.calle,
                    ciudad = d.ciudad,
                    departamento = d.departamento,
                    codigoPostal = d.codigoPostal,
                    pais = d.pais
                }).ToList()
            }).FirstOrDefaultAsync();

        if (persona == null)
        {
            return NotFound();
        }

        return persona;
    }

    // PUT: api/Persona/5  -> recibe un DTO, no la entidad (así no se sobrescribe el password)
    [HttpPut("{id}")]
    public async Task<IActionResult> PutPersona(int id, PersonaUpdateDTO dto)
    {
        var persona = await _context.Personas.FindAsync(id);
        if (persona == null)
            return NotFound();

        persona.nombre = dto.nombre;
        persona.apellido = dto.apellido;
        persona.edad = dto.edad;

        await _context.SaveChangesAsync();
        return NoContent();
    }

    // POST: api/Persona
    // To protect from overposting attacks, see https://go.microsoft.com/fwlink/?linkid=2123754
    [HttpPost]
    public async Task<ActionResult<PersonaReadDTO>> PostPersona(PersonaCreateDTO dto)
    {
        var persona = new Persona
        {
            nombre = dto.nombre,
            apellido = dto.apellido,
            correo = dto.correo,
            edad = dto.edad,
            password = BCrypt.Net.BCrypt.HashPassword(dto.password) // ahora sí: hash
        };

        _context.Personas.Add(persona);
        await _context.SaveChangesAsync();

        var resultado = new PersonaReadDTO
        {
            idPersona = persona.idPersona,
            nombre = persona.nombre,
            apellido = persona.apellido,
            correo = persona.correo
```

```
        };

        return CreatedAtAction(nameof(GetPersona), new { id = persona.idPersona }, resultado);
    }

    // DELETE: api/Persona/5
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeletePersona(int? id)
    {
        var persona = await _context.Personas.FindAsync(id);
        if (persona == null)
        {
            return NotFound();
        }

        _context.Personas.Remove(persona);
        await _context.SaveChangesAsync();

        return NoContent();
    }
}
```

PASO 8 Probar con Scalar

1. Ejecuta el proyecto con el perfil **https** (▶ en Visual Studio). Se abre `https://localhost:7186/scalar/v1`.
2. **Registro**: abre `POST /api/Auth/registro` → **Test Request** → en *Body* escribe:

```
{
  "nombre": "Ana",
  "apellido": "Pérez",
  "edad": 21,
  "correo": "ana@upc.edu.co",
  "password": "ClaveSegura2026*"
}
```

Send → `200` con los datos de la persona, sin contraseña.

3. **Login**: abre `POST /api/Auth/login` y envía:

```
{ "correo": "ana@upc.edu.co", "password": "ClaveSegura2026*" }
```

La respuesta trae `"token": "eyJhbGciOi..."`. **Copia el valor**, sin las comillas.

4. **Probar sin token**: abre `GET /api/Personas` → **Send** → `401 Unauthorized`.
5. **Pegar el token**: en la ventana de prueba, en la sección **Auth/Authentication**, verás *Bearer* seleccionado. Pega el token en el campo **Token**, sin escribir la palabra `Bearer`.
6. **Probar con token**: **Send** otra vez → `200 OK` con la lista de personas.
7. `GET /api/Auth/yo` → muestra el id, el correo y el nombre que trae el token.

Resultados esperados:

Prueba	Resultado
Registro con correo nuevo	`200`
Registro con correo repetido	`409`

Prueba	Resultado
Login con contraseña incorrecta	`401` "Credenciales inválidas."
`GET /api/Personas` sin token	`401`
`GET /api/Personas` con token	`200`
Token con una letra cambiada	`401`
Token vencido (pon `ExpiresMinutes: 1` y espera)	`401`

> ⚠ Las personas que ya estaban en tu base de datos tienen la contraseña en texto plano y no podrán iniciar sesión. Regístralas de nuevo con `/api/Auth/registro`.

Opcional: el archivo `WebApi2026New.http` trae las mismas pruebas para ejecutarlas desde Visual Studio.

WebApi2026New.http

```
@host = https://localhost:7186

### 1. Registro (público)
POST {{host}}/api/Auth/registro
Content-Type: application/json

{
  "nombre": "Ana",
  "apellido": "Pérez",
  "edad": 21,
  "correo": "ana@upc.edu.co",
  "password": "ClaveSegura2026*"
}

### 2. Login (público)
# @name login
POST {{host}}/api/Auth/login
Content-Type: application/json

{
  "correo": "ana@upc.edu.co",
  "password": "ClaveSegura2026*"
}

###
@token = {{login.response.body.$.token}}

### 3. Sin token -> 401
GET {{host}}/api/Personas

### 4. Con token -> 200
GET {{host}}/api/Personas
Authorization: Bearer {{token}}

### 5. ¿Qué trae mi token?
GET {{host}}/api/Auth/yo
Authorization: Bearer {{token}}
```

Problemas frecuentes

Síntoma	Solución
La API no arranca: *"Jwt:Key debe tener al menos 32 caracteres"*	Revisa `appsettings.Development.json`: nombre exacto, misma carpeta que `appsettings.json` y clave larga
Scalar no muestra el campo para el token	Falta el bloque `AddOpenApi(options => ...)` del Paso 5; recompila y recarga la página
Con token sigue dando `401`	Pegaste el token con comillas o con la palabra `Bearer`; `UseAuthentication()` está después de `UseAuthorization()`; o el token venció (haz login otra vez)
El login siempre da `401`	El usuario se creó antes del cambio (contraseña en texto plano); regístralo de nuevo
Error de compilación en `AddPreferredSecuritySchemes`	Tu versión de Scalar es antigua; actualiza `Scalar.AspNetCore` o quita esa línea y elige *Bearer* a mano en Scalar
Error de compilación en `OpenApiSecurityScheme` u `OpenApiComponents`	Revisa que esté `using Microsoft.OpenApi;` y que `Microsoft.AspNetCore.OpenApi` sea la versión 10

Siguiente nivel (opcional)

Cuando esto funcione, se pueden agregar:

- **Roles** (`Usuario` / `Admin`) con `[Authorize(Roles = "Admin")]`.
- **Propiedad**: que cada persona solo vea sus propias direcciones, comparando el claim `sub` con el id.
- **Correo único** con un índice en la base de datos (requiere `Add-Migration` y `Update-Database`).
- **Refresh tokens** para renovar sin volver a escribir la contraseña.

Entregable

Sube al aula virtual un único archivo PDF con:

1. Captura de Scalar con `POST /api/Auth/registro` respondiendo `200`.
2. Captura de `POST /api/Auth/login` con el token en la respuesta.
3. Captura de `GET /api/Personas` **sin token** respondiendo `401`.
4. Captura de `GET /api/Personas` **con token** respondiendo `200`.
5. Captura de la tabla `Personas` en SQL Server donde se vea la contraseña guardada como hash (empieza por `$2a$`).
6. El enlace a tu repositorio con los cambios.

Preguntas de comprobación

Responde en el mismo PDF. Justifica cada respuesta y agrega la captura cuando se pida.

1. **La clave secreta.** En `appsettings.json` la `Key` está vacía y en `appsettings.Development.json` tiene un valor. ¿Por qué se organiza así? Borra temporalmente la `Key` de `appsettings.Development.json`, ejecuta la API y **captura** el mensaje que aparece. ¿Qué podría hacer alguien que conozca tu clave?
2. **Qué hay dentro del token.** Pega tu token en un decodificador de JWT (por ejemplo jwt.io) y **captura** el resultado. ¿Qué datos de la persona puedes leer? ¿Por qué se pueden leer sin conocer la clave? ¿Por qué entonces nunca debe ir la contraseña dentro del token? Compara lo que ves con la respuesta de `GET /api/Auth/yo`.
3. **Autenticación no es autorización.** Ana (id 5) inicia sesión y ejecuta `DELETE /api/Personas/9`. ¿Qué responde tu API y por qué `[Authorize]` no lo impide? Explica con tus palabras qué tendría que comprobar el método `DeletePersona` usando el claim `sub` para que cada persona solo pueda borrar su propio registro, y qué código de estado debería responder cuando no es el dueño.
4. **Expiración.** Tu token dura 60 minutos. Si alguien lo roba 5 minutos después del login, ¿durante cuánto tiempo puede usarlo? Cambia `ExpiresMinutes` a `1`, haz login, espera dos minutos y llama a `GET /api/Personas` con ese token. **Captura** la respuesta y explica por qué una expiración corta es más segura.
5. **Correos repetidos.** ¿Qué pasaría en el login si en la tabla `Personas` quedaran dos registros con el mismo correo y contraseñas distintas? El registro ya lo revisa con `AnyAsync`, pero la base de datos no lo impide. ¿Qué agregarías al modelo `Persona` para que el correo sea único, y qué comandos ejecutarías después (`Add-Migration` y `Update-Database`)?